NAME- Divyansh Joshi
Reg No- 22BCE11364

Course → CSE2003 (Computer Architecture and organisation
Dr. Anand Motwani (C11+C12+C13)

Q1) 1-Address instruction

•) memory : Efficient as only 1 address operand required

•) Instruction Length : Shorter length due to single address specification

•) Speed : Slower as multiple instruction for complex

•) Complexity : High, additional instruction req.

2-Address Instruction

•) memory : Moderate, as it overwrite one operand

•) Instruction : Moderate, Both source & destination specified

•) Speed : Moderate, fewer step than one overwrites

•) Complexity : Moderate, reduce intermidiate steps but adds overwriting.

3 Address Instruction

-) memory usage → High, needing space for 3 specified operand
-) Instruction Length → Long, 2 Sources and 1 destination specified
-) Speed of Execution → fast, allow Complex operation in fewer steps
-) Code Complexity → low, as each instruction is self Contained and doesn't overwrite

format	Memory	Instruction Length	Speed ~~Slower~~	Complex
1- Address	Lowest	Short	Slower	High
2	moderate	Moderate	moderate	Moderate
3	Highest	Longest	fastest	Lowest

Q2) (part-a)

given → 32 bit instruction, 98 instruction
6 addressing modes

To determine Constraints and bit allocation

opcode field (Instruction) →
↳ Min bit $\log_2(98) \approx$ 7 bits ($2^6 = 64$, $2^7 = 128$)

↳ Allocate 7 bit to accomodate all 98 instruction

Mode field (Addressing) →
↳ Min bit $\log_2(6) = 3$ bit ($2^3 = 8$ ↳ modes)

↳ Allocate 3-bit for addressing mode

Total 7 (opcode) + 3 (mode) = 10 bits

remaining 32 - 10 = 22 bits

·) 0-operand → no additional bits (NOP eg)
·) 1-operand → Allocate 22 bits (register or immediate)
·) 2-operand → (eg Add)
option → ① 11 bit / operand
② 14 bit for one, 8 for symmetric

3-operand instruction (eg Add wit 3 register)

option → ① 7-8 bit per operand

② 16 bit for 2 operand

③ Any other

this might limit the range but will support all operand within 32 bit

(Part 2) →

Addressing Modes and thier best Match

① Indirect Addressing (x) → Pointer 2

↳ Dynamic Memory Access:
Refrence memory location holding address

↳ flexibility:
Support Complex Data structure (Linked List)

Advantages

↳ Dynamic Data structure: facilitate creation and manipulation

↳ Memory Efficient: Reduce hard coded address.

(2) Immediate Addressing (Y) - Best match with constants

↳ Direct operand Access: operand Specified

↳ Simplicity: Ideal for initializing variable with fix value

Advantages ↳

Speed: faster Access to contant

overhead: Embadding Contants directly

(3) Auto dicrument Addressing (Z) - Best matched with loops(1)

↳ Efficient Loop Control: Automatic after each ~~dicrument~~ operation

↳ Streamlined: Execution is smooth for Arrays and list

Advantage

↳ Code Compactness: fewer instruction

↳ Performance: Enhance Exe Speed in loops

Addressing mode	Best Match
X	Pointer
Y	Constant
Z	Loops

Q3) Converting Both no. to IEEE 754
32 bit

(1.5) first no. →

↳ Sign bit : 0 (+ve)

↳ Binary $(1.5)_{10} = (1.1)_2$

↳ normalized $= (1.1)_2 = 1.1 \times 2^0$

↳ Exponent : 1 + bias (127) = 128
$= 1,0,0,0,0,0,0,0_2$

↳ Mantissa 10000 ... 0
fractional part after binary
1 followed by 22 zeros to make 23 bit

0 10111111 10000 00000000000000

(2.75) Second no. →

↳ Binary $2.75 = (10.11)_2$

↳ normalize 1.011×2^1

↳ sign bit 0 (+ve)

↳ Exponent 1+127 = 128 (10000000)

↳ Mantissa 1.011 = 011 followed by 20 zeros so 23 bit

0 10000000 01100000000000000000

Adding the mantissas

```
  01100000 0 ... 0
  01000000 0 ... 0
-------------------
  10100 followed by 18 zeros
```

normalized 1.01×2^2

new Exponent => $E+127 = 129$

binary (10000001)

final result $1.5 + 2.75 = 4.25$

01000000101000...00

(> 21 zeros

This corresponds to the decimal value of 4.25 after conversion back from IEEE format

(14) (a) 2K x 16

2k = 2 x 1024 = 2048

Bits / word = 16

(1) Address line = $\log_2(2048)$ = 11 address line

(2) Input-output data lines = 16 data lines

(3) Memory Size => 2048 x 16 = 32768

$\frac{32768}{8}$ = 4096 bytes = 4KB

(b) 64k x 8

64x1024 = 65536 Bits/word = 8

Address line = $\log_2(65536)$ = 16

I/O data line = 8

Memory size => $\frac{65536 \times 8}{8}$ = 524288

$\frac{524288}{8}$ = 64KB

(c) 16M x 32

16M x 1024 x 1024 = 16 7 7 7216

Bits per word = 32

(1) Address lines $\log_2(16777216) = 24$

(2) I/O data lines = 32 data lines

(3) Memory Size = $\frac{16777216 \times 32^4}{8} = 67108864$

= 64 MB

(d) 46 x 64

46 = 4 x 1024 x 1024 x 1024 = 42949 67296

Bits/word = 64

(1) Address line => $\log_2(42949\,67296) = 32$

(2) I/O data line = 64 data line

(3) Memory Size => $\frac{42949\,67296 \times 64^8}{8} = 34359738368$ bytes

= 32GB

(Q5) Comparision Table SRAM-DRAM

SRAM – Static Random Access Memory
DRAM – Dynamic Random Access Memory

Charachtersticks	SRAM	DRAM
Power	Low (no refresh)	High (periodic refresh)
SPEED	fast (10-50 ns access time)	Slower (50-100 ns)
Cost	High (₹/bit)	Low (₹/bit)
Volatility	Non Volatile	Volatile
capacity	Less	More
Density	Low	High
Use	Cache memory	Main memory

Influence Usage

(1) Cache-memory : SRAM fast access time and low power makes it ideal
(2) main memory : DRAM high density and low cost makes suitable for main memory
(3) Embedded system : SRAM low power & volitality suitable for IOT devices

(4) Gaming Console : DRAM high bandwidth and low cost makes it suitable

(5) Server & Datacenter : DRAM high capacity & low cost

Trade off →

Speed vs Cost : SRAM faster speed comes at higher cost

Power vs density : Dram higher density come with more power

Volitality vs Cost : SRAM non volitatity comes at higher cost

∴ The decision btw SRAM & DRAM depends upon on trade off btw performance, cost and power consumption for high speed low latency SRAM is preferred

for large capacity storage with lower cost is preferred